COMPSCI 701:
Creating Maintainable Software

Lecture 12.1: Case Study — Encapsulating Collections

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Admin
  - Assignment 3 preliminary feedback
  - Questions?
- Case Study in improving design (encapsulating collections)
- Bonus: Debrief of lecture 5.3 class exercise

Remember Code Smells

- Code smell - characteristic of design that indicate potential code or design quality (usually maintainability) problems
- *Heuristics* — not hard and fast rules but just an indication that there may be a problem and so requires a closer look
- Remove design smells by applying refactoring (assuming you decide they need to be removed)
- From Martin Fowler “Refactoring: Improving the design of existing code” Addison-Wesley, 1999 (Design Smell chapter co-written with Kent Beck)

The problem with heuristics

- Duplicated Code
- Long Method
- Large Class
- Long Parameter List
- Speculative Generality — adding parameters or other ways to generalise a class/method/design without the actual need for the generality
- Data class — almost no methods other than getters and setters for fields
- Comments as deodorant — explaining in comments something that could be made more clear in code by refactoring the code

The problem with heuristics

- Duplicated Code — how much duplication? how to remove it?
- Long Method — how long is too long? what code to remove
- Large Class — how long is too long? which methods to remove?
- Long Parameter List — how long is too long? which parameters to remove
- Speculative Generality — adding parameters or other ways to generalise a class/method/design without the actual need for the generality — what if it's easy to do?
- Data class — almost no methods other than getters and setters for fields — what if I know I'll need more methods later?
- Comments as deodorant — explaining in comments something that could be made more clear in code by refactoring the code — ok so I'll just not put in any comments in and that will fix it right?
- One difficulty with heuristics is knowing when they apply
- Another difficulty is knowing exactly what to do

Design Smell: Unencapsulated Collection

```
public interface Book {
  String getSeries();
  String getAuthor();
  String getTitle();
}
```

```
  public static void main(String[] args) {
    List<Book> library = loadBooks(); // sorted by author
    System.out.println("There are " + library.size() +
            " books in the collection");
    String authorSearch = "John Ronald Reuel Tolkien";
    System.out.println("Books by '" + authorSearch + "' are:");
    for (Book book: library) {
       code omitted
    }
    String seriesSearch = "The Lord of the Rings";
    System.out.println("Books in series '"+seriesSearch+"' are:");
    code omitted
    String keywordSearch = "Lord";
    System.out.println("Books with the word '" + keywordSearch +
            "' in their details are:");
     code omitted #1
  }
```

Unencapsulated Collection


```
omitted code #1

String keywordSearch = "Lord";
System.out.println("Books with the word '" + keywordSearch +
                   "' in their details are:");
for (Book book: library) {
  if (book.getAuthor().indexOf(keywordSearch) >= 0 ||
      book.getTitle().indexOf(keywordSearch) >= 0 ||
      book.getSeries().indexOf(keywordSearch) >= 0) {
      System.out.println("'" + book.getTitle() + "' by " +
        book.getAuthor() + " (series '" +
        book.getSeries() + "')");
  }
}
```

Client Cohesion

```
public static void main(String[] args) {
  List<Book> library = loadBooks(); // sorted by author
  System.out.println("There are " + library.size() +
          " books in the collection");
  String authorSearch = "John Ronald Reuel Tolkien";
  System.out.println("Books by '" + authorSearch + "' are:");
  for (Book book: library) {
    code omitted
  }
  String seriesSearch = "The Lord of the Rings";
  System.out.println("Books in series '"+seriesSearch+"' are:");
  code omitted
  String keywordSearch = "Lord";
  System.out.println("Books with the word '" + keywordSearch +
          "' in their details are:");
  code omitted #1
}
```

- How long is this method?
- What else might it have to do?

Unencapsulated Collection

- Often the context schema involves a collection of items (deck of cards, books, students, etc)
- Usually we do not expect to use the full set of collection methods on our collection (e.g. deck of cards should, once created, never change; would never expect to use `toArray()`)
- Representing these context-schema-specific collections with general-purpose collections (e.g. `java.util.List`) means:
  - the collection can be changed in ways inconsistent with the context schema — a possible source of faults
  - client code (that uses the collection) must provide the code to do any necessary processing — possibly leading to duplication
  - client cohesion is reduced
  - the implementation decision to represent the context schema collection with a general purpose collection is not hidden
  - $\Rightarrow$ no encapsulation
  - Such a use of a collection is called an unencapsulated collection—See G. De Ruvo, E. Tempero, A. Luxton-Reilly and N. Giacaman 'Unencapsulated Collection — A Teachable Design Smell' *SIGCSE Technical Symposium*, February 2018. pp. 332-337. `https://doi.org/10.1145/3159450.3159469`
- So, **Encapsulate the collection**
- (Not to be confused by Fowler's Encapsulated Collection, which just involves using getters and setters — `https://martinfowler.com/bliki/EncapsulatedCollection.html`)

Unencapsulated Collection Smell Identification

- General Collections (`List`,`Collection`, `ArrayList`, arrays, etc) returned by methods, used as types of fields, and so on

```
public static void main(String[] args) {
  List<Book> library = loadBooks(); // sorted by author
  System.out.println("There are " + library.size() +
          " books in the collection");
  String authorSearch = "John Ronald Reuel Tolkien";
  System.out.println("Books by '" + authorSearch + "' are:");
  ....
```

- These collections are used in a context schema-specific way—look at where and how they are used

```
omitted code #1
String keywordSearch = "Lord";
System.out.println("Books with the word '"+keywordSearch+"' in their details are:");
for (Book book: library) {
  if (book.getAuthor().indexOf(keywordSearch) >= 0 ||
      book.getTitle().indexOf(keywordSearch) >= 0 ||
      book.getSeries().indexOf(keywordSearch) >= 0) {
      System.out.println("‘" + book.getTitle() + "’ by " +
        book.getAuthor() + " (series '" +
        book.getSeries() + "')");
  }
}
```

Encapsulating Collections

1. Create a class to represent the context schema-specific collection
2. Make the unencapsulated collection a field of the new class
3. Move the (relevant parts of the) code that does the processing of the list into methods for the new class
4. Adjust the client code to use the new class

$\Rightarrow$ refactoring

Steps 1–3

```
public class BookCollection {
  private List<Book> _library;
  public BookCollection() {
    _library = loadBooks();
  }
  public void reportCollectionSize() {
    System.out.println("There are " + _library.size() +
      " books in the collection");
  }
  public void authorSearch(String authorSearch){ code omitted }
  public void listSeries(String seriesSearch){ code omitted }
  public void keywordSearch(String keywordSearch) {
    System.out.println("Books with the word '" + keywordSearch +
      "' in their details are:");
    for (Book book: _library) {
      if (book.getAuthor().indexOf(keywordSearch) >= 0 ||
        book.getTitle().indexOf(keywordSearch) >= 0 ||
        book.getSeries().indexOf(keywordSearch) >= 0) {
        System.out.println("'" + book.getTitle() + "' by " +
            book.getAuthor()+" (series '"+book.getSeries()+"')");
      }
    }
  }
}
```

- cohesion is improved $\Rightarrow$ comprehensibility
- encapsulation means the collection cannot be changed in un-anticipated ways $\Rightarrow$ comprehensibility
- N has increased, changes to the collection management will mostly be limited to `BookCollection` $\Rightarrow \mathcal{CR}$ reduced — alterability
- testing can be done independent of the client code $\Rightarrow$ testability

Step 4

```
public static void main(String[] args) {
  BookCollection myCollection = new BookCollection();
  myCollection.reportCollectionSize();
  String authorSearch = "John Ronald Reuel Tolkien";
  myCollection.authorSearch(authorSearch);
  String seriesSearch = "The Lord of the Rings";
  myCollection.listSeries(seriesSearch);
  String keywordSearch = "Lord";
  myCollection.keywordSearch(keywordSearch);
}
```

- smaller (and reasonable method names) $\Rightarrow$ comprehensibility

Refining Abstraction

- After the refactoring comes the refinement

```
public void keywordSearch(String keywordSearch,
        boolean inAuthor, boolean inTitle, boolean inSeries) {
  String location = "in their details are:";
  if (inAuthor) {
    if (!inTitle ||!inSeries) { location = "in author are:"; }
  }
  if (inTitle) {
    if (!inAuthor||!inSeries) { location = "in the title are:"; }
  }
  if (inSeries) {
    if (!inTitle ||!inAuthor) { location = "...etc...:"; }
  }
  System.out.println("Books with the word '"+keywordSearch+"' "+
                     location);
  for (Book bk: _library) { // bad name to fit in space
    if ((inAuthor && bk.getAuthor().indexOf(keywordSearch)>=0) ||
      (inTitle && bk.getTitle().indexOf(keywordSearch)>= 0) ||
      (inSeries && bk.getSeries().indexOf(keywordSearch)>= 0)) {
      System.out.println("‘" + bk.getTitle() + "’ by " +
        bk.getAuthor() + " (series '" + bk.getSeries() + "')");
    }
  }
}
```

Refinement


```
public static void main(String[] args) {
  BookCollection myCollection = new BookCollection();
  myCollection.reportCollectionSize();
  String authorSearch = "John Ronald Reuel Tolkien";
  myCollection.authorSearch(authorSearch);
  String seriesSearch = "The Lord of the Rings";
  myCollection.listSeries(seriesSearch);
  String keywordSearch = "Lord";
  myCollection.keywordSearch(keywordSearch, false, true, false);
}
```

- technically not refactoring because functionality has changed
- but harder to see how to make this change in functionality when dealing with masses of code
- now thinking of what the `BookCollection` abstraction should be, rather than how do manage a list of `Book`

Further refinement


```
public class BookCollection implements Iterable<Book> {
  private List<Book> _library;
  public BookCollection() {
    _library = loadBooks();
  }

  public Iterator<Book> iterator() {
    return _library.iterator();
  }
  ....
}
```

How to identify

- collection (array, list, map, other kind of generic collection) as local variable (but also parameter and return type to methods)
- use of variable involves non-trivial processing of collection
- especially if multiple different kinds of processing
- (processing does not necessarily mean iterating through the collection)

Key Points (including TDD)

- TDD can lead to successful designs I would not have necessarily have chosen
- Various questions remain as to the cost/benefit tradeoff, but I am more inclined to believe than disbelieve
- But lots of research opportunities
- A common poor design decision (aka “design smell”) is to pass around a collection of items and then processing the collection in different ways — an unencapsulated collection
- Remove smell by refactoring to encapsulate collection by creating a class with the collection as a field, and methods in the class do the different processing that is required.

Bonus Extra: Lecture 5.3 Class Exercise results

- Research Question: Is code organised as a **single function** harder to understand than the same code organised (decomposed) into **multiple functions**?
- Three "problems": **weather** (W), **days since** (D), **patterns** (P). Each have a single function (S) and multiple function version (M)
- Three measurements:
  1. time taken to examine the code — expect "S" $>$ "M"
  2. correctness of explanation ("Explain in Plain English") — expect "S" $<$ "M"
  3. correctness of behaviour questions — expect "S" $<$ than "M"
- Results:
  1. WS 196 seconds $<$ WM 249 seconds (p=0.21)
     DS 275 seconds $>$ DM 201 seconds (p=0.11)
     PS 166 seconds $<$ PM 290 seconds (p=0.06)
  2. No data
  3. WS = 2.7 $<$ WM = 3.8 (p=0.02*)
     DS = 3.7 $>$ DM = 2.7 (p=0.01**)
     PS = 4.3 $>$ PM = 3.7 (p=0.12)
- See Week 12 Module for more detail